DOCUMENT-GROUNDED SUPPORT, WITH A VOICE

ZentroMart AI Support Agent

Mrinal Subudhi | Individual submission

Repository: github.com/brindoxx/ZentroMart-AI-Support-Agent

Problem and approach

ZentroMart is a fictional Indian retailer. A useful support assistant must explain its policies, handle follow-ups and avoid inventing order outcomes. I built a text and voice application that combines Gemini conversation planning, hybrid document retrieval, deterministic order workflows and Gnani speech services. The implementation uses Streamlit and custom Python rather than an agent framework.

Design boundary: the LLM proposes language and next steps; application code controls verification, calculations and tools. A fluent answer is not evidence that a refund, cancellation or handover actually happened.

Data and architecture

The supplied synthetic dataset contains 21 knowledge documents, 9 CSVs, 1,000 orders, 1,783 order items and 244 historical logs. Evaluation material includes 52 questions and 23 conversations with 96 customer turns. Retrieval uses 154 public sections/catalog records; private customer/order tables are not embedded. The calculation reference date remains 20 October 2026, not the computer's date.

Step 1 / Establish a real LLM baseline

The prompt-only adapter sends Gemini a support-role prompt and the current question without documents, tools, history or gold labels. This is distinct from the offline wiring baseline that always clarifies. Both live comparison arms used the same requested model, `gemini-3.5-flash-lite`, and 52 questions.

Step 2 / Ground answers in documents

Ingestion preserves section IDs, source paths, status and effective dates. BM25 and normalized dense cosine search feed reciprocal-rank fusion. Policy precedence and scenario applicability determine the evidence bundle; Gemini answers from that bundle. Citation-ID guards reject invalid drafts and return a safe fallback. They do not establish semantic entailment or perfect grounding.

IMPLEMENTATION / CONVERSATION AND VOICE

Step 3 / Keep one conversation and control actions

The shared `create_agent` factory is used by the UI, CLI, evaluation and batch audio runner. Each conversation owns its memory, verification and tool state. Public retrieval resources are cached and CPU access is serialized. Gemini proposes structured actions: answer, clarify, search or enter a supported workflow. The application validates the proposal before dispatching it.

English, Hindi and Hinglish replies follow the current turn. Conversation context supports follow-ups such as identifying an item and then supplying its delivery timing. Verification and active private steps can remain local; not every message needs an LLM call. Bounded history and per-session budgets limit cost and exposure.

Private workflow boundary

Order details require an eight-digit order ID and a separate typed phone suffix or pincode in a masked field. This is synthetic mock verification, not production authentication. Verified order records and proofs are excluded from the Gemini dialogue prompt. Backend code calculates policy estimates and invokes allowlisted mock tools; it cannot issue real refunds or cancel orders.

The sidebar's demo-ticket permission only enables local mock ticket writes. A handover must still be requested and the executor must confirm creation. It does not approve a refund, notify a real team or create a ticket when toggled.

Step 4 / Use Gnani for listening and speaking

The application uses Gnani WebSocket STT and Gnani TTS model `timbre-v2.5`, with no alternate speech provider or browser speech fallback. The mic button prepares an editable transcript; voice mode submits completed speech turns, plays the agent's reply and resumes listening afterwards. This is sequential voice interaction, not full-duplex or barge-in support.

The batch runner accepts a WAV folder. A manifest can group ordered files into one conversation; otherwise each file gets an independent session. It converts 16-bit PCM mono/stereo WAVs to 16-kHz mono in memory. Current capture/input limits are 15 seconds per file; long TTS replies are chunked without silent truncation. Every result records its stage, safe failure code and whether speech was actually generated. Dependent turns are skipped after a failed conversation turn.

Safety, consent and recoverability

- Cloud processing requires explicit consent in the UI; batch processing requires `--live`. Only synthetic content and the owner's consented test voice are used.
- Voice pauses for typed verification. The batch runner blocks pending proof collection instead of processing a spoken proof as verification.
- Failed requests count toward budgets. The measured batch used no automatic retries. Authentication/quota failures stop remaining batch requests.
- Batch first replies enforce AI disclosure. A guarantee across every live UI/voice response path remains a final hardening task; a banner alone is insufficient.

Enhancements beyond the minimum

The project adds automatic reply-language switching, guided invented demo orders, per-turn source inspection, deterministic estimates and a reproducible local encoder/reranker comparison. These extras do not replace the required grounded conversation, audio runner and measured baseline comparison.

MEASURED EVIDENCE / NOT AN OVERALL ACCURACY CLAIM

Step 1 versus Step 2

Measured on 31 October 2026. Both arms completed 52/52 original-text practice questions: 104 generation attempts and zero provider/adapter errors. The RAG arm used **E5-small and grounded prompt v3**, not the later BGE configuration. The comparison is stateless and does not score the full conversation controller.

Metric	Prompt-only	E5 hybrid RAG
Decision agreement (52 cases)	32.7% (17/52)	48.1% (25/52)
Required-fact lexical coverage (49 cases)	12.2%	38.8%
Declared language agreement	86.5% (45/52)	94.2% (49/52)
Retrieval Hit@5 (47 gold-evidence cases)	No retrieval	83.0% (39/47)
Retrieval Recall@5	No retrieval	55.7%
Citation precision (35 cited RAG responses)	N/A	76.7%
Citation recall (47 cases)	0%	39.4%

RAG returned 44 accepted generated answers and **8 citation-mismatch safe fallbacks**. Decision agreement checks the chosen action, not every factual claim. Lexical coverage can miss valid paraphrases or accept misleading wording. Citation scores compare source IDs, not entailment. No forbidden-claim lexical hits occurred in eight labelled cases; that is not a general safety guarantee.

Which retriever should the final app use?

Local CPU tests kept chunking, BM25 and rank fusion unchanged. Each input variant has 47 scored questions. ASR-like variants are supplied text, not new STT outputs. No Gemini/Gnani calls were used for this experiment.

Hybrid configuration	Text Hit@5	ASR Hit@5	Warm search
E5-small	83.0%	78.7%	32 ms
BGE-M3 (selected)	87.2%	83.0%	246–271 ms
E5 + MiniLM cross-encoder	76.6%	70.2%	1.68–1.70 s
BGE + MiniLM cross-encoder	76.6%	Not run	1.80 s

BGE-M3 improved top-five hits by two questions per input variant; text Recall@5 rose from 55.7% to 57.8%. The tested multilingual MiniLM cross-encoder reranked 20 candidates but reduced coverage, so it is implemented and **disabled by default**. BGE uses 1,024-dimensional CLS-pooled dense vectors; E5 uses 384-dimensional mean-pooled vectors. Both used a 512-token maximum. Warm timings exclude downloads/loading; overlapping CPU work makes them approximate.

Voice and regression evidence

Six real recordings across three cases completed STT, agent processing and TTS: **6 Gemini generations, 12 Gnani requests, no retries and no failed/skipped turns**. The four-turn case retained item context and changed from English to Hindi before requesting an order ID. The owner listened to the generated replies and reported satisfactory speech. This is a small subjective check, not WER or a general voice-reliability score. The offline suite passed **325 tests**, including 11 batch-audio and four release-package checks.

LIMITATIONS / REPRODUCIBILITY / NEXT STEPS

What went wrong and what I learned

- **Grounding is still imperfect.** Eight citation fallbacks and missing facts remain. I would inspect wrong decisions and passage support, improve language-aware evaluation and test full conversations before claiming accuracy.
- **More components did not automatically help.** The cross-encoder was slower and worse on these labels. I kept the measured simpler configuration. NumPy exact cosine matched SciPy; a database migration was not an accuracy fix.
- **An obsolete capability instruction leaked into a reply.** The saved damage answer says lookup was not connected. Review identified the old public prompt instruction; v4 corrects it. Samples remain unchanged, and neither the historical answer benchmark nor the six-file live run was repeated after this fix.
- **Remaining voice hardening.** First-response AI disclosure needs a shared guarantee across live paths. Presentation-only date/year and grouped-ID normalization, unclear-audio recovery and category-specific model retry controls remain improvements. Never weaken verification or grounding to improve fluency.

Reproduce and inspect the solution

Obtain the course dataset separately and point `DATASET_DIR` at its extracted `kb/`, `data/`, `practice/` and `tools/` folders. Create a Python 3.11+ environment, install `requirements.txt`, copy the key-free template to `.env`, and add local Gemini/Gnani keys. The first retrieval run downloads its pinned public encoder; keys and model weights are not included in the submission.

Batch command, from the repository root:
`python run.py audio_samples --manifest audio_cases.json --live`
Add `--offline` only after the encoder is cached. Omitting `--live` runs local WAV/manifest preflight only.

Each run creates a unique output folder with `summary.json`, model usage and numbered turn directories containing reply text, reply WAV, public source chunks and safe status. Missing/failed speech is reported explicitly. The live examples are in `sample_outputs/audio_demo`; input recordings and the ordered case manifest are included with the owner's permission.

Default batch limits are 24 model generations, 30 speech requests, five-second model pacing and a conservative 250,000-input-TPM guard. The browser has its own budgets. These controls do not discover remaining project-wide account quota. The original 52-question experiment used user-reported 15 RPM / 250k TPM / 500 RPD limits and no retries; another run consumes additional quota.

Where to verify the evidence

Code and setup: github.com/brindoxx/ZentroMart-AI-Support-Agent

Answer metrics: `evals/results.md`; includes denominators, historical prompt hash and reproduction caveats.

Retriever selection: `evals/retrieval-experiments.md`; pinned model revisions, recall/MRR and local report IDs.

Recorded examples: `sample_outputs/README.md`; real outcomes, known sample wording issue and owner listening review.

Checks: `python -m unittest discover -s tests -q` and `node tests/composer_frontend.mjs`.

This is an educational synthetic-data application. No fine-tuning was performed; practice labels were not used as training data or agent context. Performance is development-set evidence, not held-out accuracy or production readiness. The final demo and clean-install check are release gates.